

Capstone Technical Design and Implementation Baseline

Midterm Practical Examination | Systems Fundamentals

SALINLAHI: A Voice-Interactive Filipino Language Learning Game
Bachelor of Science in Information Technology — Capstone Project
University of Perpetual Help System Laguna – JONELTA

Group 33

Member	Role in the technical design
Daquis, Jhesza Mhei G.	Game Logic Developer — voice command flow, objective sequencing, scoring
Lacida, Kylo Bryan	UI and Testing Lead — menus, level map, scene prompts, device testing
Manzanero, Kyla Samantha	Documentation Lead — technical documentation, data design, integration write-up
Repository	<code>github.com/c23-2075-433-droid/Group33_DatabaseConnectivity</code>
Visibility	Public — accessible to the instructor without an invitation
Engine / platform	Unity 6 (6000.6.2f1), 2D URP, C#, Android build target
Baseline commit	<code>ab8f046</code> on <code>main</code>
Date	October 2026

PART I. FINAL SYSTEM ARCHITECTURE

20 points

The diagram below is the corrected architecture resulting from the Capstone System Cohesion Check. The main correction was that scoring had been drawn as a single scene-level component, which did not match the implementation: a level spans several Unity scenes, so the score is now shown as two separate responsibilities — a per-scene counter and a level-wide running total — feeding one save operation. The second correction was the placement of the speech recognizer, which is an operating-system service outside the system boundary rather than a component the group builds.

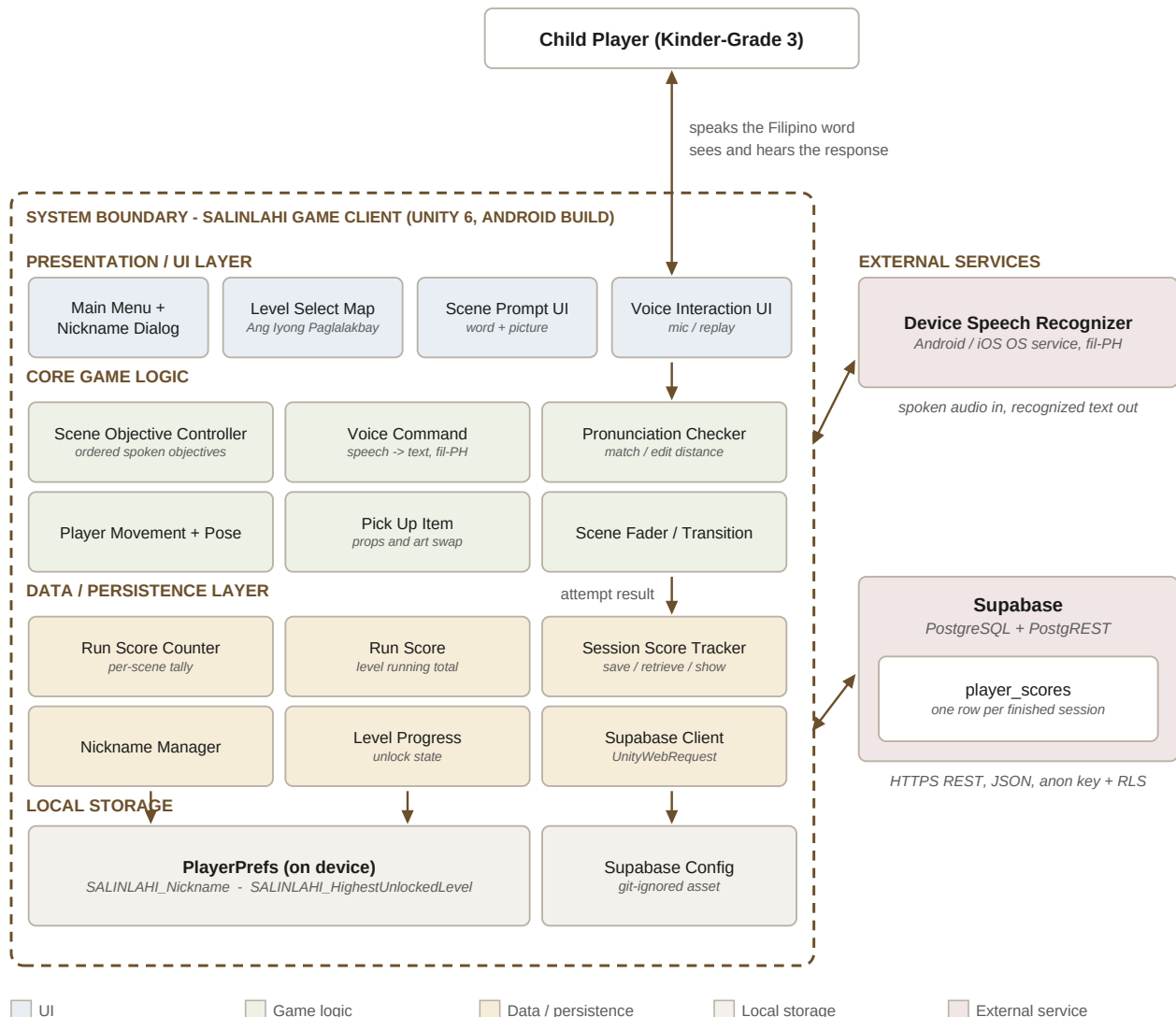


Figure 1. SALINLAHI system architecture. Dashed outline marks the system boundary; components outside it are services the group consumes but does not implement.

Architecture explanation

Overall structure. SALINLAHI is a single Unity client with a three-layer internal structure — presentation, core game logic, and a data/persistence layer — plus two external services. There is no custom backend server: the game talks directly to Supabase's REST interface, so the client is the only application the group deploys.

Where user interaction occurs. All interaction happens in the presentation layer. The child enters a nickname at the main menu, chooses an unlocked environment on the level-select map, then plays a scene where the prompt UI shows the Filipino word as text and picture and the voice interaction UI shows mic and replay buttons. The child's input is speech, not keyboard or controller.

Where major processing occurs. Processing is in the core game logic layer, inside the client. Voice Command passes captured audio to the device recognizer and receives recognized text; Pronunciation Checker decides whether that text matches the expected word, using normalisation, a per-word list of accepted variants, and a length-scaled Levenshtein tolerance, because child speech and Tagalog recognition are both noisy. Scene Objective Controller owns the ordered list of words for the scene and raises the configured effect when one is answered — moving the character, removing a prop, swapping the character art, or fading into the next scene.

Where persistent data is stored. In two places, for two different reasons. Session results are stored remotely in Supabase's `player_scores` table, because the evaluation phase needs results collected across devices and testers. Player identity and map progress are stored locally in `PlayerPrefs`, because they must work instantly and offline every time the app opens.

How components communicate. Inside the client, communication is by direct C# method calls and C# events — Voice Command raises `OnAnswerChecked`, which Run Score Counter and Scene Objective Controller both subscribe to, so the counter never has to be wired into each objective. Run Score is a static class rather than a scene object, so the running total survives scene loads within a level. Across the boundary, Supabase Client uses `UnityWebRequest` over HTTPS with JSON bodies.

External services. Two. The device speech recognizer (accessed through the `UnitySpeechToText` plugin, configured to `fil-PH`) is an OS capability available on Android and iOS only; in the Unity Editor it is unavailable, so a keyboard path feeds the identical code. Supabase provides PostgreSQL with a PostgREST API; the game authenticates with the anon public key and is restricted by Row Level Security to inserting and selecting on one table.

PART II. COMPONENT RESPONSIBILITY SPECIFICATION

15 points

Each row is a component shown in Figure 1. Responsibilities are deliberately non-overlapping: recognition, correctness, sequencing, counting, and persistence are five separate jobs held by five separate components.

Component	Responsibility	Input	Processing / Function	Output	Depends On
Main Menu Controller	Starts a session and captures the player's nickname	Nickname text, Play button	Validates the entry and stores it, then loads the level map	Saved nickname, scene change	Nickname Manager
Level Select Controller	Presents the four environments as a journey map	Taps on map nodes	Shows each node locked or unlocked and blocks locked ones	Scene load for the chosen level	Level Progress
Scene Objective Controller	Drives the ordered spoken-word objectives of one scene	Objective list (word, variants, prompt, audio, effect)	Shows the current prompt, sets the target word, advances on a correct answer, raises the per-word effect	Prompt display, correct-word events, scene-complete event	Voice Command
Voice Command	Bridges the game to the device speech recognizer	Microphone audio, current target word	Starts and restarts recognition, receives recognized text, asks Pronunciation Checker to judge it	OnAnswerChecked (text, wasCorrect)	Device speech recognizer
Pronunciation Checker	Decides whether the spoken text counts as the target word	Recognized text, target word, accepted variants	Normalises both strings, checks exact and variant matches, then a length-scaled edit distance	Correct / not correct	None (pure C# utility)
Player Movement	Moves and poses the character	Calls from objective effects	Applies movement and swaps the character's sprite set	Updated position and appearance	Unity physics
Pick Up Item	Transfers a scene prop onto the character	A correct answer for a pick-up word	Disables the prop and swaps to the matching character art set	Updated scene and character art	Player Movement
Scene Fader / Room Transition	Moves the player between scenes without a hard cut	Scene-complete or exit trigger	Fades to black, loads the next scene by name, fades back in	Next scene loaded	Unity SceneManager
Run Score Counter	Counts this scene's voice attempts into the level total	OnAnswerChecked events	Adds one attempt, and one correct answer when applicable, to Run Score	Updated running total	Voice Command, Run Score
Run Score	Holds one playthrough's totals across scene loads	Attempt results	Keeps correct and attempt counts as static values; reset at level start and after a save	Correct count, attempt count	None (static class)
Session Score Tracker	Turns a finished level into a saved record and shows it back	Scene-complete event, Run Score totals, nickname	Builds the record, requests the save, then requests recent records and writes them into the completion panel	Saved record, on-screen score and recent results	Supabase Client, Run Score, Nickname Manager
Supabase Client	Performs the actual database calls	Record to insert, row limit to fetch	POSTs one row and GETs recent rows over HTTPS, attaching the anon key headers	Success / error, list of records	Supabase Config, Supabase REST API
Supabase Config	Supplies connection credentials without committing them	Project URL and anon key entered locally	Stores them in a git-ignored asset read at runtime	Endpoint and key	Unity Resources
Level Progress	Remembers which environments are unlocked	Level completion	Raises the highest unlocked level and persists it	Unlock state for the map	PlayerPrefs

Component	Responsibility	Input	Processing / Function	Output	Depends On
Nickname Manager	Remembers who is playing	Nickname from the main menu	Caches and persists the nickname, with a friendly default	Nickname for display and for the saved record	PlayerPrefs

PART III. DATA / DATABASE DESIGN

15 points

SALINLAHI uses a relational database for session results and local key-value storage for player identity and progress. The split is deliberate: evaluation data must be collected centrally, while progress must work offline.

IMPLEMENTED NOW - SUPABASE (POSTGRESQL)

player_scores		
player_id	uuid	PK
player_name	text	not null
score	integer	not null
attempts	integer	not null
level	text	not null
remarks	text	
created_at	timestampz	default now()

Current design is a single denormalised session table: every row carries the nickname, so no join is needed to show recent scores.

IMPLEMENTED NOW - LOCAL (PLAYERPREFS)

PlayerPrefs (device key-value store)	
SALINLAHI_Nickname	string
SALINLAHI_HighestUnlockedLevel	int

Written by NicknameManager and LevelProgress; survives app restarts.

PLANNED - NORMALISATION FOR PER-PLAYER HISTORY

1 : N

players (not yet implemented)		
player_uid	uuid	PK
nickname	text	unique
created_at	timestampz	

player_scores.player_uid would become a FK to players.player_uid (1 : many).

Figure 2. Current data design, with the planned normalisation shown dashed.

Relational schema — Supabase (PostgreSQL)

Table: player_scores. One row is written per completed level. There is exactly one table at present; no foreign keys exist yet, which is why no join is required to display recent results.

Field	Data type	Key / constraint	Purpose
player_id	uuid	Primary key, gen_random_uuid()	Uniquely identifies the saved session record
player_name	text	Not null	Nickname of the child who played the session
score	integer	Not null, default 0	Number of Filipino words pronounced correctly
attempts	integer	Not null, default 0	Total voice attempts made, correct and incorrect
level	text	Not null	Which level the record belongs to
remarks	text	Nullable	Readable summary, for example 3/5 words correct
created_at	timestampz	Not null, default now()	When the record was saved; used to order recent results

Local storage — Unity PlayerPrefs

Storage key	Data type	Identifier	Purpose
SALINLAHI_Nickname	string	Fixed key	The current player's nickname, read by any scene and written into each saved record
SALINLAHI_HighestUnlockedLevel	int	Fixed key	Highest level reached, used by the map to lock and unlock the four environments

PlayerPrefs is Unity's built-in per-device key-value store. It is chosen over a file or a database call because both values are tiny, needed at startup, and must not depend on connectivity.

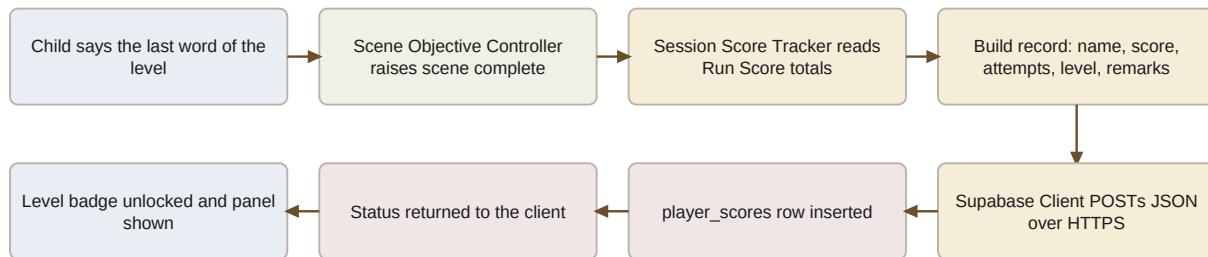
Data integrity and validation rules

Rule	Where it is enforced
player_id must be unique	Database — primary key with a generated UUID default, so the client never supplies it
player_name and level must not be empty	Database not null constraints; the client also substitutes a default nickname rather than sending a blank
Score and attempts cannot be negative	Client — both are incremented only, never decremented. A database check constraint is planned
Score must never exceed attempts	Client — a correct answer increments both counters in the same call
One record per completed playthrough	Client — the tracker refuses a second save for the same run, and resets the totals after a successful save
Records may be added and read, never changed or deleted	Database — Row Level Security grants the anon role insert and select only; no update or delete policy exists
Credentials must not enter the repository	Repository — the config asset is git-ignored and only the anon public key is ever used

PART IV. END-TO-END INTEGRATION DESIGN

15 points

Process 1 — Save session result (online path)

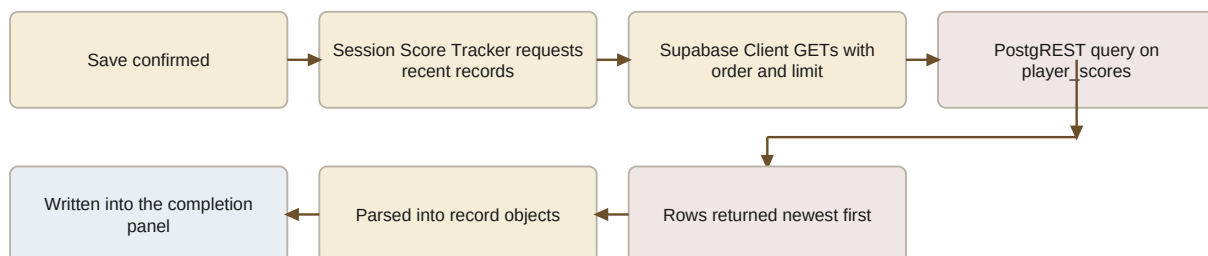


Item	Description
Trigger	The child correctly says the final spoken objective of the level
Source component	Scene Objective Controller (scene-complete event)
Data generated / sent	player_name, score, attempts, level, remarks as a JSON body. player_id and created_at are deliberately omitted so the database supplies them
Processing component	Session Score Tracker builds the record; Supabase Client performs the HTTPS POST
Destination	Supabase player_scores table, plus a local PlayerPrefs write to unlock the next level
Stored / retrieved data	One inserted row, echoed back by the API so the client can confirm what was stored
Expected response	HTTP 2xx with the inserted row; the client treats any other code as a failure
User output	The level-complete panel shows this run's score, and the next node on the journey map becomes available

Failure handling

Failure	What happens
No internet connection on the device	The request fails and the error is logged. The completion panel still shows the run's score from Run Score, and the level still unlocks, because unlocking is local. The session result is lost rather than queued — an offline queue is listed as planned work
Credentials missing or invalid (401)	The client detects the missing config before sending and reports a clear message naming the config asset instead of failing silently
Insert rejected by Row Level Security (403)	Treated as a save failure; the panel still displays the local score so play is never blocked
Scene complete fires twice	The tracker saves only once per run, so a repeated event cannot create duplicate rows

Process 2 — Retrieve and display recent results



Item	Description
Trigger	A successful save; the fetch runs immediately afterwards so the child sees the result in context
Source component	Session Score Tracker
Data generated / sent	A GET request with <code>select=*, order=created_at.desc</code> and a row limit (five by default)
Processing component	Supabase Client, which attaches the anon key headers and parses the JSON array
Destination	Back into the level-complete panel in the game UI
Stored / retrieved data	The five most recent session records, newest first
Expected response	HTTP 200 with a JSON array; an empty array is valid and means no sessions have been saved yet
User output	The completion panel lists the recent results underneath this run's score

Failure handling

Failure	What happens
Request fails or times out	The recent-results area is left empty and the error is logged; this run's own score is unaffected because it comes from Run Score, not from the fetch
Empty array returned	Not an error. It means nothing has been saved yet, and the panel simply shows no previous results
Record is missing expected fields	Unity's JSON parser leaves them at their default values, so a malformed row renders as a blank entry rather than crashing the panel

PART V. REPOSITORY AND IMPLEMENTATION STRUCTURE

10 points

Repository name	Group33_DatabaseConnectivity
Repository link	https://github.com/c23-2075-433-droid/Group33_DatabaseConnectivity
Visibility	Public
Development platform	Unity 6 (6000.6.2f1), 2D Universal Render Pipeline, C#
Target platform	Android (device speech recognition); Unity Editor used for development with a keyboard fallback
README status	Present and current — covers setup, database configuration, scene build order and controls
.gitignore status	Present — excludes <code>Library/</code> , <code>Temp/</code> , <code>Logs/</code> , <code>builds</code> , IDE files, and the credentials asset
Git LFS	Not used. The largest committed assets are PNG sprites well under GitHub's limit, so LFS would add setup cost without benefit
Branching approach	Single main branch with small, descriptive commits. Short-lived feature branches are planned once parallel work on separate chapters begins

Project structure

```
SALINLAHI/  
  Assets/  
    Scenes/      7 scenes: MainMenu, LevelSelect, Chapter 1 Levels 1-4, Chapter 2 Level 1  
    Scripts/     25 runtime C# scripts (all architecture components in Figure 1)  
    Editor/      24 editor scripts: scene builders and project tooling  
    Resources/   SupabaseConfig.asset (git-ignored) + example file  
    Sprites/     116 imported sprites: characters, props, backgrounds, UI  
    Audio/V0/    voice-over clip slots (recording not yet done)  
    Fonts/  
  Art/          raw source artwork before import processing  
  Docs/         SupabaseSetup.md, VoiceOverScript.csv, reports  
  Tools/        Python asset-processing scripts  
  Packages/     manifest.json, including the speech-to-text plugin  
  ProjectSettings/  
  README.md
```

There is no Prefabs/ folder. Scenes after the first are generated by the editor scripts in Assets/Editor rather than assembled by hand, which keeps scene composition reviewable as code in version control instead of as binary scene diffs.

Where each architecture component lives

Architecture component	Repository location
Main menu, nickname, level map UI	Assets/Scripts/MainMenuController.cs, NicknameManager.cs, LevelSelectController.cs
Scene prompt and voice UI	Assets/Scripts/SceneObjectiveController.cs, VoiceInteractionUI.cs
Voice recognition and matching	Assets/Scripts/VoiceCommand.cs, PronunciationChecker.cs
Character behaviour and props	Assets/Scripts/PlayerMovement.cs, PoseSwitch.cs, PickupItem.cs
Scene transitions	Assets/Scripts/SceneFader.cs, RoomTransition.cs, ExitDoorTrigger.cs
Scoring	Assets/Scripts/RunScore.cs, RunScoreCounter.cs, SessionScoreTracker.cs
Database access	Assets/Scripts/SupabaseClient.cs, SupabaseConfig.cs, ScoreRecord.cs
Local persistence	Assets/Scripts/LevelProgress.cs, NicknameManager.cs
Game scenes	Assets/Scenes/
Scene construction tooling	Assets/Editor/Build*.cs
Database schema and setup	Docs/SupabaseSetup.md

PART VI. IMPLEMENTATION BASELINE AND DEVELOPMENT PLAN

10 points

Current technical baseline

Component / Feature	Current state	Evidence	Next action	Assigned
Voice recognition and word matching	Implemented	VoiceCommand.cs, PronunciationChecker.cs; 22 words recognised on device	Tune accepted variants from real child-speech testing	Daquis
Objective sequencing	Implemented	SceneObjectiveController.cs; objectives configured per scene	Reuse unchanged for new chapters	Daquis
Chapter 1 (Bahay), Scenes 1-4	Implemented	Chapter1_Level1-4 scenes; playable end to end	Device playtest with target-age children	Lacida
Chapter 2 (Paaralan), Scene 1	Implemented	Chapter2_Level1_Paaralan.unity; Kumusta, Pasok, Takbo	Build Scenes 2-3	Daquis
Score aggregation across scenes	Implemented	RunScore.cs, RunScoreCounter.cs	Add a counter to every new scene as it is built	Daquis
Save and retrieve session results	Implemented	SupabaseClient.cs; player_scores table with RLS policies	Add a check constraint for non-negative values	Manzanero
Level unlock and journey map	Implemented	LevelProgress.cs, LevelSelect.unity, four level badges	Wire Parke and Palengke once their scenes exist	Lacida
Nickname capture and persistence	Implemented	NicknameManager.cs, nickname dialog on the main menu	Map the nickname to a player record	Manzanero
Voice-over audio	In progress	Docs/VoiceOverScript.csv complete; Assets/Audio/VO slots wired but empty	Record and import the clips	Lacida
Results display	In progress	Recent records shown as plain text on the completion panel	Format as a readable leaderboard	Lacida
Per-player score history	Planned	Single denormalised table; no player entity yet	Introduce a players table and filter by player	Manzanero
Offline save queue	Planned	Saves currently fail silently without connectivity	Queue failed saves locally and retry on reconnect	Daquis
Parke and Palengke chapters	Planned	Locked nodes on the map; artwork drafted	Design the word list for each environment	Group
Evaluation instrument	Planned	Objective 5 of Chapter 1; no instrument prepared yet	Prepare the functionality and usability questionnaire	Manzanero

Next three technical milestones

Order	Technical milestone	Dependency / what must exist first	Expected output
1	Record and integrate the voice-over audio	The completed recording script and the existing instruction-clip slots in Scene Objective Controller	Every objective speaks its word aloud, so pre-readers can play without reading the prompt
2	Introduce a player entity and filter results per player	Milestone 1 is independent, but the schema change must be agreed and the nickname must map to a stable identifier	A players table, a foreign key from player_scores, and a completion panel showing that child's own history
3	Build Paaralan Scenes 2 and 3 and unlock Parke	Milestone 1, so new scenes are authored with audio from the start, plus the existing scene-builder pattern	Chapter 2 playable end to end and the third map node reachable

The order follows actual technical dependencies rather than convenience. Voice-over comes first because every scene built after it would otherwise have to be revisited to add audio. The data model change comes before more content because changing a schema is cheaper while only one table holds records. New scenes come last because they consume both of the earlier results.

PART VII. TECHNICAL DEMONSTRATION / EVIDENCE

5 points

The following can be shown live from the repository and the running project:

- The public repository and its commit history on the main branch.
- The project structure in Unity, matching the tree in Part V.
- An implemented component end to end: saying a Filipino word advances the scene and increments the score.
- The source of that component — `VoiceCommand.cs`, `PronunciationChecker.cs` and `SceneObjectiveController.cs`.
- The database: the `player_scores` table in Supabase with rows from real play sessions, and the Row Level Security policies in `Docs/SupabaseSetup.md`.

Status is distinguished throughout Part VI as Implemented, In Progress, or Planned. Nothing marked Implemented depends on work that has not been committed.

References

Supabase. *REST API (PostgREST) documentation*. <https://supabase.com/docs/guides/api>

Supabase. *Row Level Security*. <https://supabase.com/docs/guides/database/postgres/row-level-security>

Unity Technologies. *UnityWebRequest scripting reference*.

<https://docs.unity3d.com/ScriptReference/Networking.UnityWebRequest.html>

Unity Technologies. *PlayerPrefs scripting reference*. <https://docs.unity3d.com/ScriptReference/PlayerPrefs.html>

yasirkula. *UnitySpeechToText plugin*. <https://github.com/yasirkula/UnitySpeechToText>